Whack-A-Mole Final Project Report

ENG EC 311 Logic Design

Professor Ajay Joshi

Declan Lacey, Namig Mirzayev, Paul Nguyen, Catherine Tran, and Jonathan Uruchida

1 Objective

Our objective was to implement the game “Whack-a-Mole” onto a Nexys A7 FPGA board. When the game is opened, players are presented with a selection of an easy, medium, or hard game difficulty, represented by three distinctly colored blocks that correspond to the left, middle, and right pushbuttons. Players are then given a five-second countdown until the start of the game and 30 seconds to score as many points as possible. The countdown timer and game duration are displayed on the 7-segment display. The moles were represented on a 640 by 480 pixel VGA display as five colored blocks, which light up one-at-a-time randomly. The player must “whack” a mole by pressing the push button that corresponds to the lit “mole.” The VGA and 7-segment display update the player’s score in live-time until the game concludes, returning the player to the gamemode selection screen.

2 Design Choices

2.1 Display Method

Alongside using LEDs to display the mole on the FPGA, we chose to implement the VGA output as the main visual interface (Fig. 1). We decided to go with this route since the VGA output gives a significantly larger and more visually appealing display area, allows us to use different colors to differentiate the difficulty levels, and showcases our understanding of FPGA design.



Figure 1. VGA display of Whack-A-Mole.

2.2 Modular Design (Multiple Subfiles)

The project follows a modular structure where each functional block is implemented as a separate Verilog module. By doing this, we are able to improve code readability and make it easier to debug each game component. The top-level module *whack_game_top.v* instantiates and connects all sub-modules while handling clock signal communication between components.

3 System Architecture and Module Hierarchy

The system consists of eleven Verilog modules organized in a three-tier hierarchy. *whack_game_top.v*, our top module, connects and instantiates all game components and manages the input and output signals needed to play the game.

At the core logic level, we created the following modules:

- *whack_game_core.v* - Game state machine and logic controller
- *vga_whack_display.v* - VGA graphics generation and rendering
- *sevenseg_time_score.v* - 7-segment display controller

Finally, at the utility level, we implemented:

- *debouncer.v* - Button input conditioning
- *clock_divider.v* - Clock frequency division
- *counter16.v* - 16-bit counter for random number generation
- *seven_segment_decoder.v* - 4-bit hexadecimal input to 7-bit display value conversion
- *vga640x480.v* - VGA timing signal generation

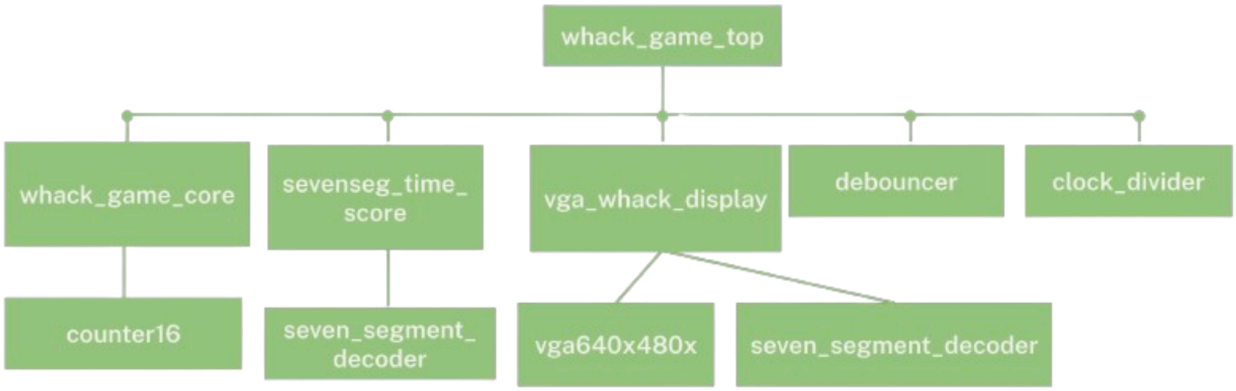


Figure 2. Block diagram of module hierarchy

4 Detailed Module Descriptions

4.1 Top-Level Module (*whack_game_top.v*)

Our top module works as the main integration point for the whole system. It manages all external I/O connections to the FPGA board and instantiates all the big subsystems. The module takes in the 100MHz clock of the system, the active-low reset button, and the five pushbuttons (*BTN[4:0]*) on the FPGA as inputs. It produces outputs for the 7-segment display (*SEG[6:0], AN[7:0], and DP*), the LEDs, and VGA signals (*VGS_HS, VGS_VS*, and 4-bit RGB color channels).

For clock generation, the top module instantiates two clock divider modules to generate slower clock signals from the system's 100Mhz clock. The first divider produces a 1Hz clock by dividing the input frequency by 50,000,000. This clock drives the countdown timer and the 30-second game timer. The second divider generates a 10Hz clock by dividing by 5,000,000, which controls how often the moles

appear and allows for different difficulty levels by adjusting how many of the 10Hz ticks pass before a new mole appears.

For button debouncing, all 5 of the physical pushbuttons are processed through their own debouncer modules instantiated with a generate block. Since mechanical buttons have rapid oscillations between high and low states when pressed or released, the debouncer plays an important role in keeping the input signals of the button clean. Debouncer modules produce a single-cycle pulse on a valid button press to produce the clean signal. Without the debouncing, a button press could register as multiple inputs and ruin the gameplay.

4.2 Game Logic (*whack_game_core.v*)

The core module is the main processing module/component of the game and implements a finite state machine to control all of the gameplay logic. The module directs the gameflow from level selection screen through gameplay to the game over stage and redirection to the level select screen. The core module FSM has four states with defined transitions and responsibilities.

The states include:

1. S_LEVEL_SELECT (3'd0) - This level select state gives the players to select between three difficulty options of easy, medium, and hard. The module takes inputs from three specific buttons: *btn_hit[3]* for easy mode, *btn_hit[0]* for medium mode, and *btn_hit[4]* for hard mode. When the button is pressed the corresponding difficulty level is stored in the level register and the state transitions to S_COUNTDOWN.

2. S_COUNTDOWN (3'd1) - This countdown state begins the 5-second countdown until the beginning of the actual gameplay. The module monitors the *sec_tick* signal, which pulses once every second based on the 1Hz clock. On each tick, the countdown value decreases by 1. When the countdown reaches 0, the state transitions to S_RUNNING and the *mole_tick* signal is set to 0.

3. S_RUNNING (3'd2) - The actual gameplay state is where all the main game logic/mechanisms operate at the same time. The module manages three different timing mechanisms. The game timer counts down from 30 seconds using *sec_tick* and when it reaches 0, the state transitions into S_GAME_OVER. The mole timer operates at 10Hz with the *mole_tick* and controls how long each mole stays lit on the screen. The timeout value depends on difficulty level where easy is 10 ticks/1s, medium is 7 ticks/0.7s, and hard is 4 ticks/0.4s. When the *mole_tick* signal reaches the value defined by the timeout value, a new random mole is selected. Each hit detection monitors all 5 pushbutton inputs simultaneously using a for loop that checks each button press against the current mole position. If the pressed button index matches *current_mole*, then the score increments, the mole timer resets, and a new mole appears. Incorrect button presses are ignored.

4. S_GAME_OVER (3'd3) - After the game ends, this state keeps the last screen on from the gameplay, displaying the score in the top right corner. Then, after 3 seconds, the screen automatically returns to the level selection screen. The *game_over_secs* counter tracks the delay and increments until reaching 2 and then all the game parameters reset and the FSM returns to S_LEVEL_SELECT

For the random number generation for the positioning of the mole, we used a continuously running 16-bit counter to increment on every system clock cycle. When a new mole is needed the lower 3 bits of the counter are examined. Since those bits cycle through the values of 0-7, the module uses combinational logic in the always block in *next_mole* to constrain the value to the valid range (0-4) by subtracting 5 when the value exceeds 4. Plus, the logic prevents the same mole from appearing twice consecutively by checking if *next_mole == current_mole* and incrementing to the next position if so. While this method is not perfectly random, it is more than enough for gameplay to seem natural.

The core module also handles signals crossing from slower clock domains to the main 100MHz domain. It registers the slow clock signals and uses edge detection logic to generate single-cycle pulses. This is computed as sec_tick = sec_clk & ~sec_clk_d, which produces a pulse only on the rising edge of sec_clk. By doing this, timing issues can be prevented and the countdown and mole timing events are sure to be processed exactly once per clock edge.

4.3 Seven-Segment Display (*sevenseg_time_score.v*)

The segment display module drives the 8-digit 7-segment display on the Nexys A7 board, which shows the game timer and the score simultaneously. The 8 digits are divided into two sections. Digits 4-7 (leftmost four digits) display timing information. During the countdown state, these show the countdown value (0-5 seconds). During running and game-over states, they show the remaining game time (0-30 seconds). Since the maximum time value is 30, only the 10s and 1s seconds digits are needed; the remaining 2 digits display zeros. Digits 0-3 (rightmost 4 digits) display the score. Digit 3 is reserved for future expansion but currently displays zero. Digits 2, 1, and 0 show the hundreds, tens, and ones places of the score (supporting values 0-255, though only 0-999 can be displayed).

The 8-bit score value has to be split into individual decimal digits for the display. We did this by using combinational logic with division and the mod operations. The hundreds digit is computed as score/100. The remainder is computed as score % 100 and stored in a temp register *s_tmp*. The tens digit is *s_tmp* / 10 and the ones digit is *s_tmp* % 10. To drive the 8 7-segment displays with limited I/O pins, the FPGA board uses multiplexing. All eight digits share the same 7-segment lines (CA-CG), but each digit has its own anode enable signal. Only one digit is active at any time, with the active digit determined by grounding its anode. A 17-bit refresh counter increments every clock cycle, and its upper 3 bits (refresh_cnt[16:14]) select which digit to illuminate. This creates eight scanning positions, cycling through all digits approximately 381 times per second ((100 MHz / 2^15)/8 ≈ 381 Hz). Since bit 14 is the 15th bit, we use 2^15 in the equation.

A large combinational case statement based on *scan_sel* activates the appropriate digit by setting one of an *an_reg* to 0 while keeping others at 1. At the same time, it selects the corresponding 4-bit digit value (*cur_digitI*) from the pre-computed digit values. This digit value is fed to the decoder instance which then converts it into the correct 7-segment pattern. The decoder outputs 7-segment patterns in standard active-low encoding where 0 means the segment is on. However, the FPGA board expects the segments in a specific bit order that differs from the decoder's natural output, so the module includes bit-reordering logic that maps *seg_raw[6:0]* to *SEG[6:0]* with reversed bit indices, making sure correct display of all digits and letters.

4.4 VGA Display (*vga_whack_display.v*)

The VGA display module makes all the graphics for the VGA monitor, and renders the game interface with the level selection boxes, the mole holes, and the score display. The module works on the pixel level and determines the color of each individual pixel based on its position and the current game state. This module instantiates the *vga640x480.v* module which works to create pixel mappings that scan across the 640x480 screen. A 25 MHz pixel clock is derived from the 100Mhz system clock using a 16-bit accumulator. The timing core outputs the active pixel coordinates (x,y) used for rendering

In S_LEVEL_SELECT, three difficulty boxes are showing using coordinate checks

- Easy: 80-200 x 120-220 (green)
- Medium: 240-360 x 120-220 (yellow)
- Hard: 400-520 x 120-220 (red)

During gameplay, 5 80x80 mole holes are drawn in a cross patterned centered at the middle of the display. The module renders active or inactive mole states using coordinate checks against the regions of each mole. For the score display, the VGA screen shows the two least-significant score digits as large, rendered 7-segment numerals. Each digit is drawn by mapping segment patterns from two of the seven segment decoder modules to rectangular pixel regions in the upper-right corner of the screen.

4.5 Reusable Modules

debouncer.v - Based off of the module in Lab 2, we created debouncing modules for each of the physical push buttons on the FPGA. Push buttons make electrically noisy signals when pressed, with the signal bouncing between high and low states before stabilizing, so the debouncer clears that noise and cleans the signal for a clean input. In the debouncer, a 20-bit counter increments while the button is pressed. When the counter reaches its maximum value, and a single-cycle pulse is generated on *button_out*, and the output flag is set.

clock_divider.v - Also based on a module from Lab2, the clock divider module generates slower clock signals from the fast system clock by implementing a frequency divider. It maintains a 32-bit counter that increments each input clock cycle. When the counter reaches the first division parameter, the output clock toggles and the counter resets. For a 1Hz output from the 100MHz global clock input, a division by 50,000,000 is used, meaning the output toggles every 50 million cycles and produces a 1Hz square wave. For the 10Hz mole clock, it is divided by 5,000,000.

counter16.v - Another module from Lab 2, the 16-bit counter works as the random number source for the game and continuously increments on every clock cycle when the incr input is high and wraps around the 0 after reaching 65,535.

seven_segment_decoder.v - This module, also from Lab 2, uses a lookup table that converts 4-bit hexadecimal values to 7-segment display patterns. Each case in the case statement maps an input value to its 7-segment code. The encoding is active-low, where a bit value of 0 makes the segment light up. All 16 hexadecimal values are supported, allowing for display of digits 0-9 and letters A-F, even though we only need the digits. The default case outputs 7'b1111111 (all segments off) for any invalid input.

vga640x480.v - From the project resources, we downloaded this module to work on the digital display of the game. Since the standard VGA video requires precise timing signals, this module works to properly display the game. It has two main counters: h_count for the horizontal position and v_count for vertical position. The horizontal sync pulse is made when h_count is between 16 and 112. The vertical sync pulse is made when v_count is between 490 and 492. The module outputs both sync signals and the current pixel coordinates within the active region. When pixels are outside the active area, coordinates are kept in the same ranges.

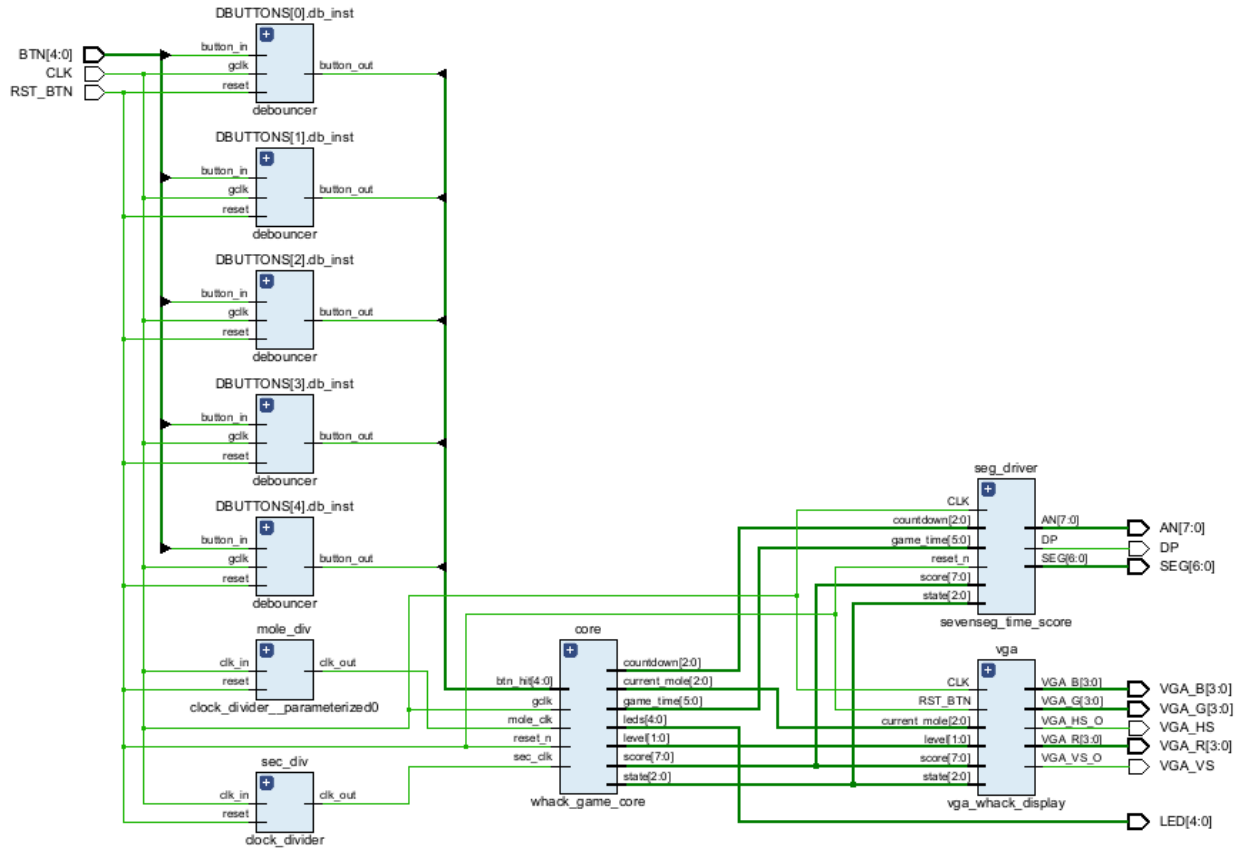


Figure 3. Elaborated Design of Whack-A-Mole

5 Timing Synchronization

Clock Domains: The system operates across multiple clock domains: the primary 100 MHz system clock drives all sequential logic, while derived 1 Hz and 10 Hz clocks control game timing. Direct use of slow clocks would create timing hazards, so the core module implements clock domain crossing using edge detection. The slow clocks are registered in the 100 MHz domain, and combinational logic detects rising edges, generating single-cycle pulses that safely trigger state changes.

VGA Timing: VGA requires precise pixel clock timing (25 MHz for 640×480 resolution). Rather than generating a true 25 MHz clock, the design uses clock enable methodology with the pix_stb signal. This strobe pulses at 25 MHz rate, enabling the VGA counters only when appropriate. This approach simplifies clock distribution while maintaining timing accuracy.

Debounce Timing: Button debouncing must filter signals fast enough to feel responsive but slow enough to eliminate all bounce. The 20-bit counter provides approximately 10ms delay, which testing showed to be optimal, short enough for responsive gameplay but long enough to handle typical mechanical switch bounce characteristics.

Display Refresh: The 7-segment multiplexing rate is determined by an 18 bit counter. The counter is incremented at the global clock frequency of 100MHz. Then scan_select register takes the three most significant bits of this number (enough for 8 anodes) and uses it to select the 7-segment anode to turn on via a multiplexer. The VGA display scans through all of the pixels (640x480) at a frequency of 100MHz, which is quick enough to create smooth images.

6 Testing and Validation

We created simulation files and generated waveforms for our clock divider, debouncer, counter, and seven segment controller modules. An example waveform using our simulation for *counter16.v* can be found in Figure 4.

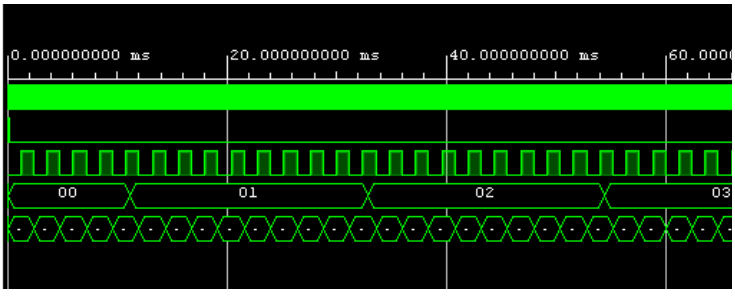


Figure 4. Waveform for simulation of 16-bit counter.

The *whack_game_core.v* module implementation did not have a dedicated test bench and was simply run with the top module to determine if the gameplay functionality was correct. In order to make debugging easier, specific states were isolated and tested individually.

7 Design Challenges and Solutions

Our team encountered several challenges related to collaboration throughout the programming process. Web-based platforms such as Github simplify parallel development for programmers, but we did not take advantage of this during our game's development. Instead, our group members would meet in-person and work on the code at the same time, catching bugs and uploading or downloading an updated version of the source files through Google Drive. Although this method worked, use of platforms similar to Github would have streamlined the overall process.

It was very difficult to initially break down the game into states in *whack_game_core.v*, which was crucial to the operation of our other design files. Our biggest obstacle was figuring out how to display the player's score, the number of whacked moles, on the VGA. Eventually, we accomplished this with the mod operator, division operator (explained in more detail in 4.3), and use of a large case mapping statement in the VGA display file.

8 Conclusions

While the premise of the Whack-A-Mole itself is simple, implementing this game with our available hardware and software was a greater challenge. We were able to create a simple rendition of the game that uses the pushbuttons on a Nexys A7 FPGA to whack the moles (lit squares) onscreen when they appear out of a hole (dimmed squares). The player's score updates with each mole whacked, both on the VGA and the three rightmost seven-segment display digits. The four leftmost digits display the time left until the game starts or ends. In between games, players can use the pushbuttons to change their difficulty mode, which changes the colors of the blocks onscreen and the speed at which the moles appear from a random hole.

Our team encountered several challenges in regards to simultaneous coding, synchronizing the FPGA inputs to the VGA display outputs, and dividing the clock signal frequency between the mole randomization, input response, etc. Although we were able to implement a working version of Whack-A-Mole, we would like to make improvements to the game, starting with pixel-art graphics and a clean user-interface (UI) to make the game more visually pleasing. We would also like to add sounds indicating whacked or missed moles to enhance the game experience. To increase the competitiveness of the game, we could also add a penalty system for missed moles or wrongly-hit holes and a leaderboard that tracks previous players' scores.

Many of us have programmed games before, but incorporating a hardware element with the FPGA and VGA was a newer skill that required us to combine what we have learned this semester. Overall our group enjoyed being able to implement what we learned throughout this semester into our final project.